UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 01

Presentación del Curso

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

Bienvenida

Bienvenido a **Programación Avanzada**. Este curso es una inmersión profunda en C++, uno de los lenguajes más relevantes y demandantes de la industria tecnológica. Aquí no solo aprenderás sintaxis: entenderás *por qué* el computador hace lo que hace, y tendrás control directo sobre ello.

> ✔ **Recuerda**
>
> Lo que distingue a este curso de otros de programación es el énfasis en **entender la máquina**. Aprenderás a gestionar memoria, a diseñar funciones eficientes y a pensar como lo haría el compilador.

Syllabus

- **Unidad 1 — Fundamentos del lenguaje C++**
  - Introducción a C++: historia, compiladores y entornos de trabajo
  - Tipos de datos, variables y operadores
  - Estructuras de control: `if`, `switch`, `while`, `for`
  - Funciones: declaración, definición, ámbito y recursión
- **Unidad 2 — Memoria y apuntadores**
  - Modelo de memoria: stack y heap
  - Apuntadores y referencias
  - Paso por valor, por referencia y por apuntador
  - Arreglos y aritmética de apuntadores
  - Memoria dinámica: `new` y `delete`
- **Unidad 3 — Programación orientada a objetos**
  - Clases y objetos
  - Encapsulamiento, herencia y polimorfismo
  - Constructores, destructores y sobrecarga de operadores
  - Clases abstractas e interfaces
- **Unidad 4 — Estructuras de datos y algoritmos**
  - Algoritmos de ordenamiento: Bubble Sort, Quick Sort, Merge Sort
  - Búsqueda lineal y binaria
  - Introducción a la STL: `vector`, `map`, `set`

Cómo se califica

La calificación final se compone de tres partes iguales:

Porcentaje	Rubro	Descripción
33 %	Tareas y trabajos en clase	Ejercicios, cuadernos y proyectos entregados
33 %	Asistencia y participación	Presencia, contribución en discusiones y actividades
33 %	Examen	Evaluación escrita y/o práctica por unidad

La participación vale tanto como el examen. Las preguntas en clase, los comentarios en las discusiones y tu presencia activa construyen el 33 % de tu calificación desde el primer día. No hay forma de recuperar esos puntos al final del semestre.

Las tareas se entregan en la fecha acordada. Cada día de retraso puede reducir la calificación de ese entregable. Consulta la política específica de tu profesor al inicio del semestre.

Bibliografía recomendada

• Libros principales

1. The C++ Programming Language — Bjarne Stroustrup
4.ª edición, Addison-Wesley, 2013.
El libro definitivo del creador del lenguaje. Denso y técnico, pero la referencia más completa que existe. Ideal para consultar cuando quieras entender *por qué* algo funciona así en C++.
https://www.stroustrup.com/4th.html

2. A Tour of C++ — Bjarne Stroustrup
3.ª edición, Addison-Wesley, 2022.
Versión condensada del anterior. Recorre el lenguaje completo en ∼300 páginas. Excelente para tener una visión panorámica rápida.
https://www.stroustrup.com/tour3.html

3. C++ Primer — Lippman, Lajoie, Moo
5.ª edición, Addison-Wesley, 2012.
El libro introductorio más utilizado en cursos universitarios de C++. Paso a paso, con muchos ejemplos y ejercicios. Muy recomendado si es tu primer contacto formal con el lenguaje.

4. The C Programming Language — Kernighan & Ritchie
2.ª edición, Prentice Hall, 1988.
El legendario “K&R”. C++ nació de C, y entender C ilumina muchas decisiones de diseño de C++. Clásico imprescindible para cualquier programador serio.

- **Recursos en línea (gratuitos)**
  - **cppreference.com** — `https://en.cppreference.com`
    La referencia técnica más completa y actualizada del lenguaje y la STL.
  - **learncpp.com** — `https://www.learncpp.com`
    Tutorial gratuito muy completo, con ejercicios. Excelente complemento a las clases.
  - **C++ Core Guidelines** — `https://isocpp.github.io/CppCoreGuidelines`
    Guía de buenas prácticas escrita por Stroustrup y Herb Sutter. Aprende no solo a que el código funcione, sino a que sea correcto.
  - **Compiler Explorer (Godbolt)** — `https://godbolt.org`
    Herramienta online que muestra el ensamblador generado por tu código. Fascinante para entender qué hace realmente el compilador.

¿Por qué C++ sigue siendo relevante en 2026?
Según el índice TIOBE, C++ se mantiene consistentemente entre los 3 lenguajes más usados del mundo. Es el lenguaje dominante en videojuegos (Unreal Engine), sistemas embebidos, motores de bases de datos, navegadores web y software de alto rendimiento. El mercado laboral para programadores C++ ofrece algunos de los salarios más altos en la industria.
Referencia: `https://www.tiobe.com/tiobe-index/cplusplus/`

Actividades de la sesión

- **Padlet — ¿Qué emoción te produce programar?**

En el Padlet del curso encontrarás un tablero compartido. Tu tarea es **agregar un post-it** con un meme, GIF o imagen que represente cómo te sientes cuando programas.

No hay respuesta correcta o incorrecta. La intención es romper el hielo y conocernos como grupo.

Padlet del curso:

`https://padlet.com`

(El enlace exacto lo comparte el profesor al inicio de clase)

Los mejores programadores no son los que no sienten frustración — son los que aprendieron a convertir la frustración en curiosidad. Si tu meme es de frustración total, bienvenido: estás en el lugar correcto.

- **Lluvia de ideas — ¿Qué se necesita para ser un buen programador hoy?**

Reflexiona sobre estas preguntas antes de compartir tus ideas con el grupo:

- ¿Qué habilidades técnicas consideras esenciales?
- ¿Qué habilidades *no técnicas* crees que importan más de lo que la gente piensa?
- ¿Cómo cambia la respuesta si consideramos herramientas de IA como GitHub Copilot, ChatGPT o Claude?
- ¿Qué diferencia a alguien que *usa* un lenguaje de alguien que realmente lo *domina*?

Mis notas de la discusión:

• Ejercicio de mecanografía — 10FastFingers

Uno de los factores que más impacta tu flujo de trabajo como programador es la **velocidad y precisión al escribir**. No se trata de escribir rápido a expensas de pensar — se trata de que tu velocidad de escritura no sea el cuello de botella cuando ya tienes la solución clara en la cabeza.

Plataforma:

`https://10fastfingers.com`

Modo: Español — Test de 1 minuto

Registra tu resultado aquí:

Métrica	Mi resultado
Palabras por minuto (PPM)	
Precisión (%)	
Fecha	

¿Cuánto es normal?
El promedio de un adulto es ∼40 PPM. Un programador experimentado suele estar entre 60–80 PPM. Mecanógrafos profesionales superan los 100 PPM. La mayoría de las personas mejoran 10–15 PPM en pocas semanas de práctica deliberada. Lo importante no es el número de hoy, sino la tendencia al final del semestre.
Referencia: Dhakal et al. (2018). "Observations on Typing from 136 Million Keystrokes". *CHI 2018*. https://doi.org/10.1145/3173574.3174220

Las partes del computador — La analogía del escritorio

Antes de escribir una sola línea de código es fundamental entender con qué estamos hablando. Un computador tiene partes muy distintas con roles muy distintos. La forma más intuitiva de verlas es compararlas con un **escritorio de trabajo físico**.

Parte del computador	Analogía	Qué hace
CPU	El usuario sentado	Ejecuta instrucciones, una a enorme velocidad. Es qu sa" y procesa.
RAM (memoria)	El espacio sobre el escritorio	Todo lo que estás usan *mismo* vive aquí. Es rápi mitada: cuando cierras el se borra.
Almacenamiento(disco duro / SSD)	Los cajones del escritorio	Guarda información de f manente. Mucho más len RAM pero no se borra al
Bus de datos	Las manos del usuario	Transporta datos entre la RAM y el almacenamient
Sistema operativo	Las reglas de la oficina	Administra quién usa el cuánto espacio tiene cad ma y cómo acceden a los
Programa en ejecución	La tarea sobre el escritorio	El conjunto de papeles instrucciones (pasos a se el usuario está procesand momento.

• El flujo de trabajo

Cuando abres un programa (por ejemplo, tu navegador), el sistema operativo **saca su código de los cajones** (almacenamiento) y lo coloca **sobre el escritorio** (RAM) para que el usuario (CPU) pueda trabajar con él. Cuantas más cosas tienes abiertas,

más abarrotado queda el escritorio. Si se llena, el sistema empieza a usar los cajones como extensión del escritorio — eso se llama *memoria virtual* y es mucho más lento.

✔ Recuerda

En C++ tú decides qué va al escritorio (RAM) y cuándo se quita. Eso es exactamente lo que hace la gestión de memoria con `new` y `delete`. Tener ese control es la razón por la que C++ es tan rápido; también es la razón por la que requiere más cuidado que Python o Java, donde un conserje automático (el *garbage collector*) limpia el escritorio por ti.

• Velocidades: una perspectiva

La diferencia de velocidad entre las partes es enorme. Si escalamos el tiempo que tarda la CPU en ejecutar una instrucción a **1 segundo**, así se ven las demás operaciones:

Operación	Tiempo real	Escala humana
Instrucción en CPU (caché L1)	∼0.3 ns	1 segundo
Acceso a RAM	∼100 ns	∼6 minutos
SSD (lectura aleatoria)	∼100 µs	∼4 días
Disco duro mecánico	∼10 ms	∼1 año
Red (mismo país)	∼50 ms	∼5 años

Esta tabla de perspectiva fue popularizada por el ingeniero Jeff Dean de Google y es conocida en la industria como “Latency Numbers Every Programmer Should Know”. Los valores han cambiado con el tiempo pero las proporciones se mantienen. Conocer estas magnitudes guía decisiones de diseño: cuándo cachear datos, cuándo usar una base de datos vs memoria, etc.
Referencia: `https://gist.github.com/jboner/2841832`

• Relevancia para este curso

Toda esta sesión suena abstracta hoy. Volverá a tener sentido concreto cuando veamos **apuntadores** (sesión 14): en ese momento estaremos hablando literalmente de celdas numeradas del escritorio (RAM) y de cómo mover datos entre ellas sin que el conserje intervenga.

Lo que viene

En la **Sesión 02** comenzamos con C++ directamente:

- Instalación y verificación de `g++` en tu equipo
- Primer programa: `Hello World`
- Compilar y ejecutar desde la consola
- Apertura del cuaderno interactivo en Jupyter con xeus-cling

✔ Recuerda

Para la próxima sesión necesitas tener acceso a una terminal con `g++` instalado **o** acceso a Google Cloud Shell (no requiere instalación, funciona en cualquier navegador).